Lab Assignment 13.5

Name: K. Charan Yadav

Hall Ticket No: 2303A52367

Batch No: 20

Task 1: Refactoring – Removing Global Variables

Prompt:
Refactor a legacy Python script that uses global variables for interest calculation. The goal is to eliminate global variables and pass required values as parameters or through a configuration structure. The refactored code should improve modularity, testability, and maintainability while keeping the same output behavior.

```python
# Task - 1 : Refactoring – Removing Global Variables
# No global variables are used in this code.
# The rate is already passed as a parameter to the function.
# The code is modular and testable.
# If you want to further improve modularity, you could use a configuration dictionary:

def calculate_interest(amount, config):
    return amount * config['rate']
config = {'rate': 0.1}
print(calculate_interest(1000, config))
```

Output :

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding> &
100.0
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:
Global variables make programs harder to test and maintain because multiple functions can modify shared data unexpectedly. Passing variables through parameters makes the code more modular and predictable. This improves code readability and allows easier unit testing and reuse of functions in different contexts.

Task 2: Refactoring Deeply Nested Conditionals

Prompt:

Refactor a Python program that uses deeply nested if–else statements for evaluating student scores. The objective is to simplify the logic using a cleaner structure such as guard clauses, loops, or mapping-based approaches. The new design should improve readability, maintainability, and flexibility.

```python
# Task - 2 : Refactoring Deeply Nested Conditionals (Dynamic Version)
def evaluate_score(score, thresholds):
    for threshold, label in thresholds:
        if score >= threshold:
            return label
    return "Needs Improvement"

# Define thresholds dynamically
thresholds = [
    (90, "Excellent"),
    (75, "Very Good"),
    (60, "Good"),
]
score = int(input("Enter the score: "))
print(evaluate_score(score, thresholds))
```

Output :

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
Enter the score: 95
Excellent
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:

Deeply nested conditional statements make code difficult to read and modify. Flattening the logic using structured approaches such as lists or mappings improves readability and reduces complexity. It also makes it easier to update grading criteria without modifying large sections of code.

Task 3: Refactoring Repeated File Handling Code

Prompt:

Refactor a Python program that repeatedly opens, reads, and closes multiple files. The goal is to remove duplicated file handling logic by creating a reusable function that uses Python's context manager (with open()). The function should accept the filename as a parameter and safely handle missing files.

```python
# Task - 3 : Refactoring Repeated File Handling Code
def read_and_print_file(filename):
    try:
        with open(filename, 'r') as file:
            print(file.read())
    except FileNotFoundError:
        print(f"File '{filename}' not found.")

read_and_print_file("data1.txt")
read_and_print_file("data2.txt")
```

Output:

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
File 'data1.txt' not found.
File 'data2.txt' not found.
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:

Repeated code violates the DRY (Don't Repeat Yourself) principle. Using a reusable function with a context manager improves maintainability and ensures that files are properly closed after use. This also reduces the chance of file handling errors and makes the code cleaner and easier to extend.

Task 4: Optimizing Search Logic

Prompt:

Optimize a Python program that searches for a username within a list using a loop. Refactor the solution to use a more efficient data structure such as a set or dictionary to improve search performance. The program should check whether a username exists and display access control messages.

```python
# Task - 4 : Optimizing Search Logic
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
print("Access Granted" if name in users else "Access Denied")
```

Output:

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding> &
Enter username: admin
Access Granted
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:
Linear search in lists requires O(n) time complexity. Using a set allows membership testing in O(1) average time complexity. This significantly improves performance for large datasets and demonstrates the importance of selecting appropriate data structures for efficient algorithms.

Task 5: Refactoring Procedural Code into OOP Design

Prompt:
Refactor a simple procedural Python program that calculates employee salary and tax into an object-oriented design. Create a class called EmployeeSalaryCalculator with attributes and methods for calculating tax and net salary.

```python
# Task - 5 : Refactoring Procedural Code into OOP Design
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self, rate=0.2):
        return self.salary * rate

    def calculate_net_salary(self, rate=0.2):
        tax = self.calculate_tax(rate)
        return self.salary - tax

salary_calculator = EmployeeSalaryCalculator(50000)
net_salary = salary_calculator.calculate_net_salary()
print(net_salary)
```

Output:

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
40000.0
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:
Object-Oriented Programming improves code organization by encapsulating data and behavior within classes. It promotes reusability, scalability, and better maintainability. Using classes also makes the system easier to extend when additional features such as bonuses or deductions are required.

Task 6: Refactoring for Performance Optimization

Prompt:

Optimize a Python program that calculates the sum of even numbers using a loop for a large range of values. Refactor the code using a mathematical formula to improve performance. Compare execution time between the legacy and optimized implementations.

```python
# Task - 6 : Refactoring for Performance Optimization
# Legacy code timing
start = time.time()
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
legacy_duration = time.time() - start

# Optimized code using formula
start = time.time()
# Sum of even numbers from 1 to 999999
# Even numbers: 2, 4, ..., 999998
n = (999998 // 2)
optimized_total = n * (n + 1)
optimized_duration = time.time() - start

print(f"Legacy total: {total}, time: {legacy_duration:.6f}s")
print(f"Optimized total: {optimized_total}, time: {optimized_duration:.6f}s")
```

Output:

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
Legacy total: 249999500000, time: 0.174893s
Optimized total: 249999500000, time: 0.000003s
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:

Loops over large datasets can significantly increase execution time. Using a mathematical formula reduces the time complexity and improves performance. Comparing execution time demonstrates the importance of algorithm optimization in high-performance applications.

Task 7: Removing Hidden Side Effects

Prompt:

Refactor a Python function that modifies a global list by appending items. The goal is to eliminate hidden side effects by designing the function to return a new list instead of modifying shared state.

```python
# Task - 7 : Removing Hidden Side Effects
def add_item(data, x):
    return data + [x]
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

Output:

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
[10, 20]
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:
Functions that modify global variables introduce hidden side effects and make program behavior unpredictable. Returning new values instead of mutating global data improves functional purity, predictability, and easier debugging.

Task 8: Refactoring Complex Input Validation Logic

Prompt:
Refactor a Python program that validates a password using multiple nested conditions. Split the validation logic into smaller reusable functions such as checking length, digits, and uppercase characters. The main validation function should combine these checks to determine password validity.

```python
# Task - 8 : Refactoring Complex Input Validation Logic
def is_long_enough(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

def validate_password(password):
    if not is_long_enough(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"

password = input("Enter password: ")
print(validate_password(password))
```

Output:

```
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding> &
Enter password: Charan9948
Valid Password
PS C:\Users\chara\OneDrive\Desktop\Ai-Assisted Coding>
```

Justification:
Breaking complex validation logic into smaller functions improves readability and testability. Each function handles a single responsibility, which follows the Single Responsibility Principle. This approach makes the validation logic easier to maintain, modify, and reuse in other applications.